

Software Engineering

Complete Examination Preparation Guide

TMC 202 · B.C.A. Semester II

Graphic Era Hill University

Department of Computer Applications

Unit 1 SDLC & Process	Unit 2 SRS & Design	Unit 3 Coding	Unit 4 Testing & SPM	Unit 5 Quality & Maint.
--------------------------	------------------------	---------------	-------------------------	----------------------------

References: Software Engineering — Pressman & Software Engineering — Ian Sommerville

45 Contact Hours · 5 Units · All Assignments Included

Unit 1 • Introduction to Software Engineering & SDLC

- ◆ Software Process & SDLC
- ◆ Waterfall Model
- ◆ Incremental Development
- ◆ Prototyping (Evolutionary & Throw-away)
- ◆ Boehm's Spiral Model
- ◆ Agile Methods
- ◆ Extreme Programming (XP)
- ◆ Scrum Process

Unit 2 • Software Requirements & Design

- ◆ Requirement Engineering Process
- ◆ Functional vs Non-Functional Requirements
- ◆ Feasibility Study
- ◆ Elicitation Techniques
- ◆ SRS Document & IEEE Standard
- ◆ Verification & Validation
- ◆ Software Design Principles
- ◆ Top-Down vs Bottom-Up
- ◆ DFDs
- ◆ Cohesion & Coupling
- ◆ UML & OOD

Unit 3 • Software Coding

- ◆ Introduction to Coding & Configuration Management
- ◆ Coding Guidelines
- ◆ Code Documentation
- ◆ Code Review: Walkthrough vs Inspection

Unit 4 • Software Testing & Project Management

- ◆ Testing Fundamentals: Error, Bug, Fault, Failure
- ◆ Verification vs Validation
- ◆ White Box Testing Methods
- ◆ Black Box Testing Methods
- ◆ Levels of Testing
- ◆ Alpha, Beta, Acceptance, Regression Testing
- ◆ Software Project Management
- ◆ LOC & Function Point Estimation
- ◆ COCOMO Model (Basic, Intermediate, Complete)

Unit 5 • Quality, Maintenance & CASE Tools

- ◆ Software Quality Concepts & Attributes
- ◆ ISO 9000 Certification
- ◆ SEI CMM (5 Levels & KPAs)
- ◆ Types of Software Maintenance
- ◆ Forward, Reverse Engineering & Re-engineering
- ◆ Software Configuration Management (SCM)

- ◆ CASE Tools (Upper, Lower, Integrated)

Practice • Assignments & Solved Problems

- ◆ COCOMO Solved Numericals
- ◆ Critical Path Method (CPM)
- ◆ Equivalence Partitioning & BVA Test Cases
- ◆ Design Principles Application Questions

UNIT
1

Introduction to Software Engineering

9 Contact Hours · SDLC Models · Agile Methods



1.1 Software & Software Engineering

DEFINITION — SOFTWARE ENGINEERING (IEEE)

The application of a systematic, disciplined, quantifiable approach to the development, operation and maintenance of software — the application of engineering to software.

PRESSMAN'S DEFINITION

The establishment and use of sound engineering principles in order to obtain economically software that is reliable and works efficiently on real machines.

Software Crisis (Historical Context)

Software Engineering formally began at the **1968 NATO Software Engineering Conference**. The trigger was the Software Crisis of the 1960s–70s:

- Projects going over budget and missing deadlines
- Poor software quality and unreliable performance
- Difficulties in maintenance and updates
- Mismatch between user requirements and final product
- Lack of proper documentation and communication

Historical examples: Therac-25 (1985–87) radiation therapy machine software bugs → patient deaths. Y2K bug — no foresight about 2-digit year scalability.

Software Characteristics

Key insight: Software is *engineered*, *not manufactured*. It does not wear out — but requires updates to adapt to new requirements.

Characteristic	Description
Functionality	Set of features and capabilities — storage, processing, UI, security, networking
Reliability	Performs functions correctly and consistently — bugs and poor algorithms cause failures

Characteristic	Description
Efficiency	Optimal use of resources (memory, CPU, bandwidth)
Usability	Ease of use — time and effort to learn the system
Maintainability	Ease of modification — extending functionality or correcting errors
Portability	Ability to transfer across environments with minimum changes (e.g., Java WORA)

1.2 Software Process

DEFINITION

A software process is a collection of related activities for producing quality software. It defines how software is produced — including the life-cycle model, tools, and individuals involved.

Four Core Activities

Activity	Description
Specification	Define functions and conditions; produce requirement specification document
Design & Implementation	Convert requirements into design; implement using programming languages
Validation	Verify software fulfills client requirements; unit, system, acceptance testing
Evolution	Continuous modification to accommodate changing needs (maintenance)

★ EXAM TIP

Process descriptions also include: Products (outcomes), Roles (responsibilities), Pre- and post-conditions. Plan-driven = fixed requirements; Agile = flexible, iterative.

1.3 SDLC Models

Waterfall Model

Developed by Royce (1970). Also called the *software life cycle*. Sequential — one phase must complete before the next begins.

Phases (in order):

1. Requirement Definition — gather client requirements
2. System & Software Design — architecture and blueprint

3. Implementation & Unit Testing — convert design to code; test units
4. Integration & System Testing — combine and test complete system
5. Operation & Maintenance — deploy; accommodate new requirements

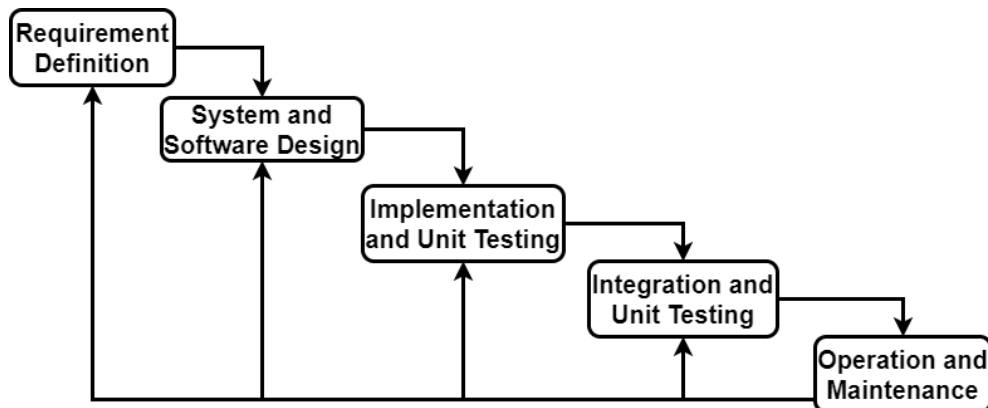


Figure: Waterfall Model — sequential phases with feedback loops

■ Advantages	■ Disadvantages
Simple and easy to understand	Rigid — difficult to accommodate mid-project changes
Each phase is well-documented	No working software until the very end
Suited for smaller systems with stable requirements	Not suitable for complex or long-ongoing projects
Progress tracked accurately	Integration done at end — bugs found late

★ BEST SUITED FOR

Government & defence software, medical/aerospace, core banking, embedded/hardware systems where requirements are clear, fixed, and well-documented. Maintenance consumes maximum effort among all phases.

Incremental Development Model

Produces an initial software, delivers it, gets user feedback, then arrives at the final version. Based on agile principles — mistakes can be corrected more easily than in Waterfall.

Three Repeated Phases:

- Specification — collect requirements; prepare system specifications
- Development — prepare design/model; complete programming
- Validation — test developed software to ensure requirements are fulfilled

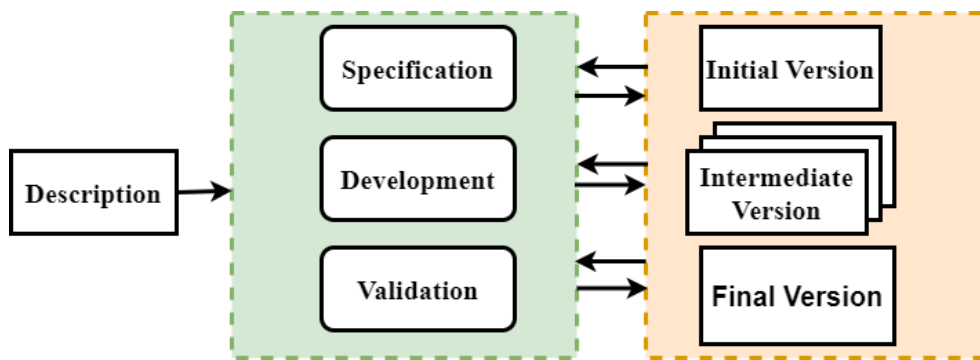


Figure: Incremental Development — iterative specification, development, and validation

Each repetition produces a new **increment/version**. Urgent work goes first; additional features added based on customer response. Uses a mix of plan-driven and agile.

■ Advantages	■ Disadvantages
Users' changes can be incorporated	Multiple versions can be costly
Most important functionality first with better quality	Frequent changes can corrupt structure
Less time; easier error detection	Clients must be involved — not always easy

Prototyping Model

DEFINITION

A prototype is a starting/initial version of a software system with limited functional capabilities, low reliability, and inefficient performance. Used to demonstrate concepts, clarify requirements, and get feedback before the final system is built.

Types of Prototyping:

Type	Description	Advantage Drawback
Evolutionary Prototyping	Prototype developed, given to clients, refined based on feedback, modified in iterations. Starts with best-understood requirements. Used for AI/ML, UI systems where specs not fully known.	Quick delivery; client involved from start Requires specialist skills; expensive maintenance
Throw-away (Rapid) Prototyping	Trial version for poorly understood requirements, given for experimentation, then DISCARDED. Actual system built using Waterfall or another model afterwards.	Fast; clarifies requirements Incomplete problem analysis; scope creep risk

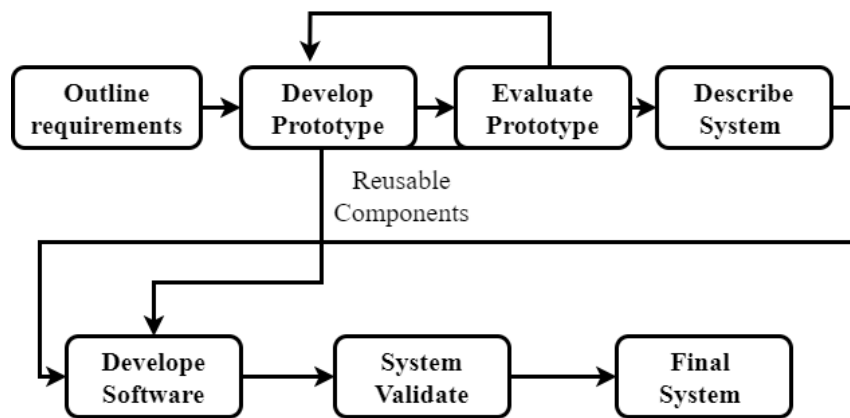


Figure: Evolutionary Prototyping — iterative development and evaluation cycle

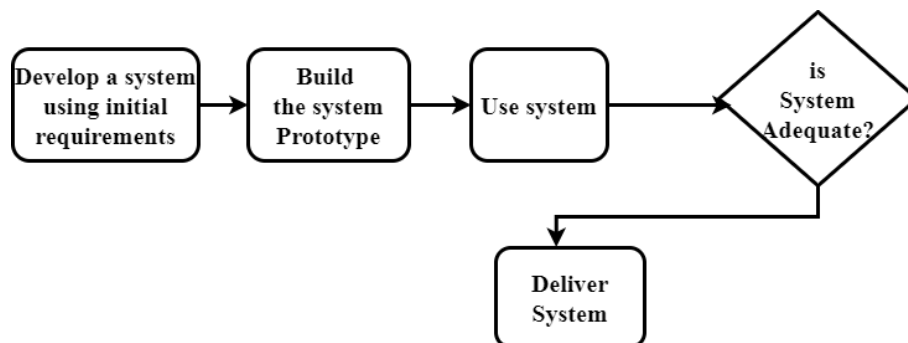


Figure: Throw-away Prototyping — prototype built and discarded; system adequacy checked before delivery

Boehm's Spiral Model

Proposed by Barry Boehm (1986). Known as the **Meta Model** — combines Waterfall, Incremental, and Evolutionary characteristics. *Risk-driven.*

Four Quadrants per Spiral:

Quadrant	Activities
Quadrant 1 — Determine Objectives	Define objectives, constraints, management plan; identify risks and alternative strategies
Quadrant 2 — Assess & Reduce Risks	Detailed risk analysis; develop prototype if required; resolve technical uncertainties
Quadrant 3 — Development & Validation	Select model based on risk evaluation; develop, test, and validate the system
Quadrant 4 — Planning	Review and decide whether to continue; plan next phases (schedule, cost, manpower)

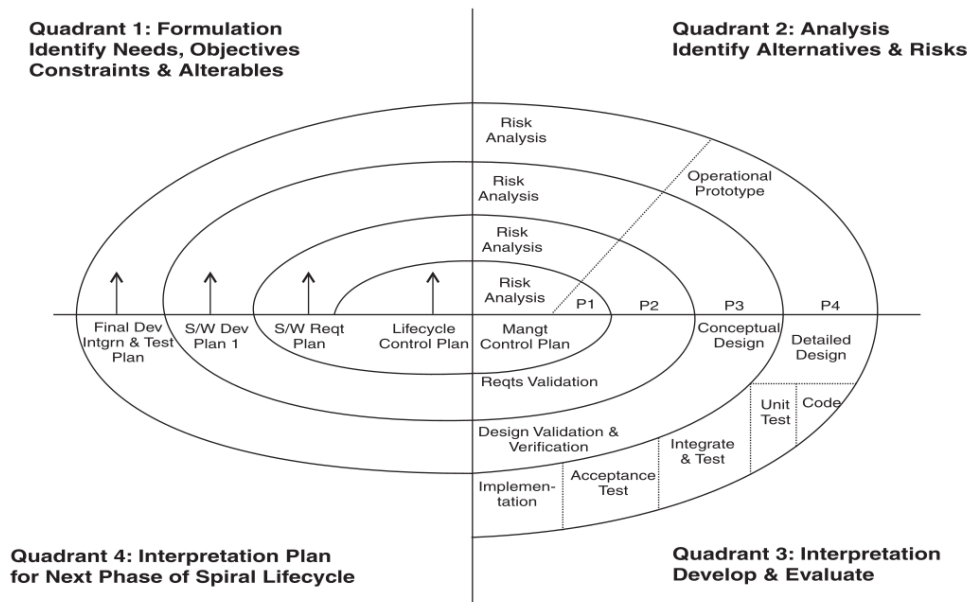


Figure: Boehm's Spiral Model — four quadrants with cumulative cost and risk analysis

■ Advantages	■ Disadvantages
Risk management features — risks identified early	Not suitable for small projects
Flexible to changing requirements	Risk assessment expertise required
Combines best features of multiple models	More documentation — time consuming
Customer satisfaction built in	May not meet schedule or budget

1.4 Agile Methods

WHY AGILE?

Plan-driven approaches had overhead — too much documentation, delays in planning. Not suitable for small/medium projects or frequently changing requirements. Agile methods emerged in early 2001.

Agile Manifesto Values

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

Agile vs Plan-Driven

Aspect	Plan-Driven	Agile
Activities	Planned with fixed, linear stages	Done in iterations, parallel fashion
Documentation	Heavy documentation at each stage	Minimal; focus on design and implementation
Requirements	Fixed before development starts	Flexible; change as needed
Best for	Large complex projects, stable requirements	Small/medium projects, changing requirements

Extreme Programming (XP)

Most well-known agile method. Called "extreme" because agile activities are pushed to an extreme level. Requirements expressed as **user stories**, implemented as tasks.

- Based on incremental development; user stories used as base for new functions
- Customers involved in acceptance testing of developed software parts
- **Pair programming:** People work in groups of two — exchange knowledge, check work
- New versions built many times per day; increments delivered every two weeks
- Simplicity maintained throughout

Extreme Programming (XP) Methodology

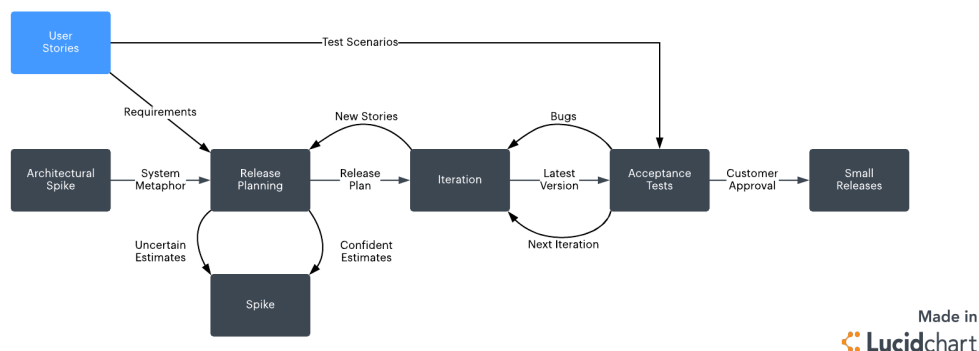


Figure: Extreme Programming (XP) Methodology — from user stories to small releases

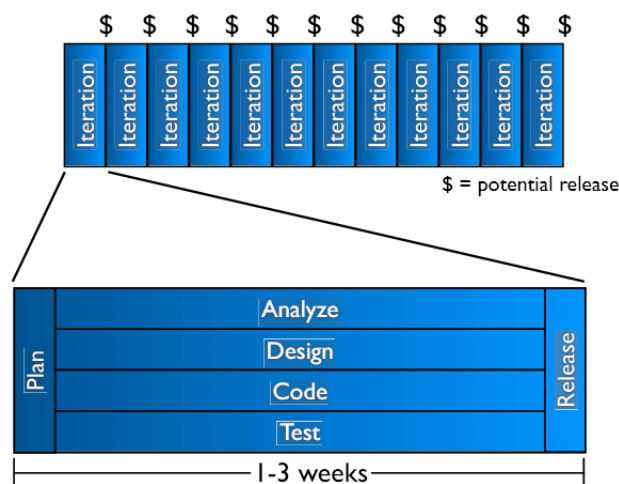


Figure: Agile Iterations — each 1–3 week iteration potentially releasable

Scrum Process

DEFINITION

A method for software projects where there is continuous experimentation and response from clients. Software developed in a series of increments called sprints (2 weeks to 2 months). Introduced 1995.

Scrum Roles:

- **Scrum Master:** Organizes and coordinates the scrum; removes obstacles
- **Development Team:** Group of developers creating the software project
- **Product Owner:** Ensures the product meets all requirements; manages product backlog

Three Phases:

1. Initial Phase: Define the product backlog — all features/work to be done
2. Sprint Cycle: Short iterations where teams develop, test, and deliver working software; each sprint has 5 events and 3 artifacts
3. Closure Phase: Wrap-up, documentation, and retrospective

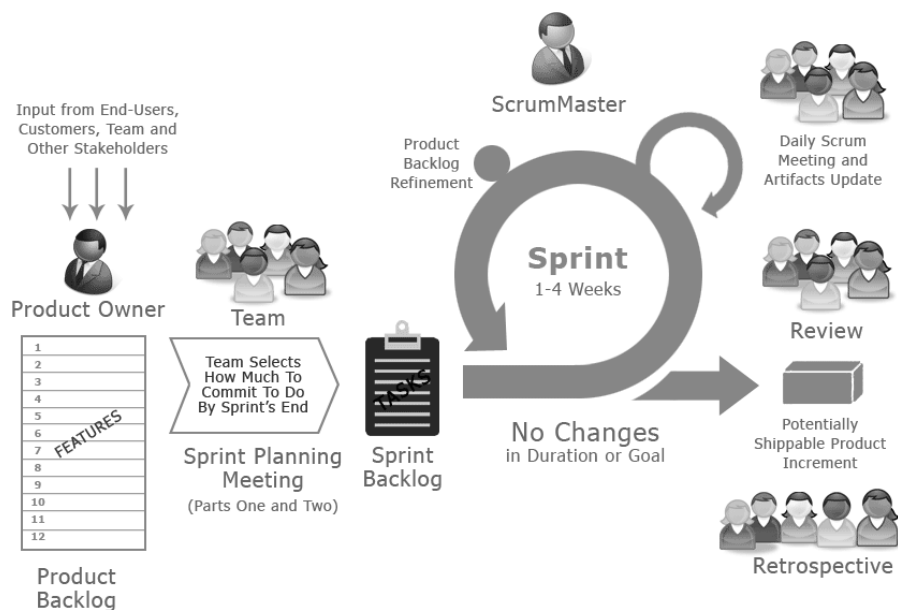


Figure: Scrum Process — Product Backlog → Sprint Planning → Sprint → Shippable Increment



Software Requirements & Design

10 Contact Hours · SRS · DFDs · Cohesion & Coupling · UML



2.1 Requirement Engineering (RE)

DEFINITION

Requirement Engineering (RE) is a process of analyzing, searching, and documenting requirements. Determines problem description, why it needs solving, possible solutions, and I/O of the system.

Types of Requirements

Type	Description
User Requirements	High-level abstract requirements for end-users and clients. Written in natural language. NOT related to software system development.
System Requirements	Detailed description of system functions, services, and operational constraints. Related to HOW to develop the system considering the client's business.

2.2 Functional vs Non-Functional Requirements

Functional Requirements

Describe **what functions** the software provides — how it reacts to inputs and generates outputs. Must be **complete and consistent**.

EXAMPLE — HOSPITAL MANAGEMENT SYSTEM

List all clinics in the city. Each clinic maintains appointment list for patients. Each clinic maintains employee list with unique codes. User can search by area, city, and specialty.



Figure: Functional Requirements example — Online Shopping System modules and sub-functions

Non-Functional Requirements

Not directly connected to functionality — related to parameters such as reliability, efficiency, performance, security, or availability. **Often more critical** — one failure can make entire software useless (e.g., ATM security).

Category	Types
Product Requirements	Efficiency (performance, space), Dependability, Usability, Security
Organizational Requirements	Environmental, Operational, Developmental
External Requirements	Regulatory, Ethical, Legal (Safety and Security)

Metrics for Non-Functional Requirements:

Property	Metric
Speed	Transactions per second, response time
Size	Number of bytes / memory chips
Ease of use	Help options, time for training
Reliability	Failure rate, availability
Robustness	Recovery time after failure, data corruption probability
Portability	Dependency on platform / OS

2.3 RE Process — Four Generic Activities

1. Feasibility Study

Conducted in the early stage. Answers: Does the system meet organizational objectives? Can it be implemented with existing technology? Can it integrate with existing systems?

Type	What it Checks
Technical	Can the product be developed using existing technology? Maintenance possible?
Operational	Can it be operated/used easily? Can operations be maintained post-deployment?
Legal	Does it fulfill legal/ethical needs — copyright, license, security laws?
Schedule	Can the project be completed within the given deadline?

Feasibility Study Activities:

1. Introduction — problem statement; background details; list of functions
2. Summary & Recommendations — customers, competitors, technical details, time estimates
3. Alternatives — current system vs. modification options; possible solutions
4. System Description — proposed solution; teams needed; equipment modifications
5. Cost-Benefit Analysis — project cost, initial investment, expected income, cash flow
6. Evaluation of Risks — legal, financial, operational risks; calculated decision
7. Conclusion — binary Yes/No decision on the new project

2. Requirements Elicitation & Analysis

Stakeholders collect information regarding domain, activities, services, performance, software/hardware conditions.

Sub-steps:

- **Requirements Discovery** — gather user and system requirements from all stakeholders
- **Classification and Organization** — group collected requirements
- **Setting up Priority and Negotiations** — resolve differences by prioritization
- **Requirements Documentation** — early draft of SRS document

Elicitation Techniques:

Technique	Description
Interviewing	Direct discussion about workplace, challenges, and constraints
Observation / Ethnography	Observe stakeholders performing daily tasks; document actual work
Scenarios	Real-life examples of how users would interact with the system
Use Cases	Graphical notations and structured text understanding user interaction
Questionnaires	Structured list of questions covering development, testing, program management
Survey	Captures responses (agree/disagree, Yes/No); closed or open-ended questions
Existing Documentation	Technical, operational, and financial details of current system; manuals
Group Discussion / Focus Group	People from similar backgrounds to understand problem domain
Prototyping	Sample/mock model expressing ideas; virtual digital model of system
Brainstorming	Group activity generating new ideas; ends with detailed requirement list

3. Requirements Specification

Documents both user and system requirements. Simple, unambiguous, well-documented. Serves as a **contract between client and software firm**.

- **Natural Language Specification** — expressive and universal but can be vague/ambiguous
- **Structured Specification** — standard form rather than free-form text
- **Graphical Notations** — UML use-case and sequence diagrams
- **Mathematical Specifications** — formal notations; unambiguous but customers struggle to understand

4. Requirement Validation

Verifies requirements fulfill the client's needs. Checks:

- **Validity** — do requirements meet real needs of system users?
- **Consistency** — no conflicting requirements
- **Completeness** — all functions and constraints defined
- **Realism** — can it be implemented within budget and technology?
- **Verifiability** — can tests be written for each requirement?

2.4 Software Requirements Specification (SRS)

DEFINITION

SRS is a systematically organized document containing all requirements for the software to be developed. The analyst collects all information, removes incompleteness and inconsistencies, and organizes into SRS. It serves as a contract between client and software firm.

Characteristics of a Good SRS

Characteristic	Meaning
Correctness	Covers all requirements of the system
Completeness	Ready version of all requirements for submission
Consistency	No ambiguity — no contradictory requirements
Concise	Compact document — no unnecessary details
Unambiguous	Only one interpretation of each requirement
Black-box view	Implementation details hidden; only functional view given
Verifiable	Requirements can be formally verified using scientific methods
Modifiable	Can easily accept changes to the system
Traceable	Design, software, and test cases can be traced for correctness
Testable	Easy to generate test cases and test plans from the document

Characteristics of a BAD SRS:

- **Over-specification** — too detailed; difficult to understand and use
- **Forward references** — uses terms not yet defined
- **Wishful thinking** — requirements that cannot be practically implemented
- **Noise** — unnecessary details included

IEEE SRS Standard Structure

1. Introduction — Purpose, Project Scope, Definitions/acronyms, References, Overview
2. Overall Description — Product perspective, features, user classes, operating environment, design constraints, user documentation, assumptions
3. Specific Requirements — Functional requirements (use cases)
4. External Interface Requirements — User, hardware, software, communication interfaces
5. Non-functional Requirements — Performance, safety, security requirements
6. Other Requirements — Database, legal, etc.
7. Appendices — Glossary, Analysis Models, TBD List

2.5 Verification & Validation (V&V;)

Aspect	Verification	Validation
Question answered	"Are we building the product right?"	"Are we building the right product?"
Checks	Output of one phase conforms to previous phase; specs, design, code checked against standards	Fully developed system conforms to its SRS document
Code execution	May NOT include program execution — static tests	Always involves execution of code
Performed by	System analysts and designers	Software development team
Techniques	Code inspections, reviews, walkthroughs	Unit testing, integration testing, system testing
Phase	Early stages — high-level and database design	After verification of the code

2.6 Software Design

Design begins after requirement analysis. While requirements specify *what* the software should do, design specifies *how* to develop the system. Design errors are costlier than coding errors — take more time to detect and correct.

Design Principles

- **Modularity** — divide system into smaller, manageable modules
- **Abstraction** — focus on what a component does, not how; hide internal details
- **Information Hiding** — internal details of a module should not be exposed to others
- **Problem Partitioning / Divide & Conquer** — break problem into smaller subproblems

- **Low Coupling + High Cohesion** — modules independent yet internally focused
- **Simplicity, Portability, Flexibility, Reusability**

Top-Down vs Bottom-Up Design

Aspect	Top-Down	Bottom-Up
Approach	General solution → divides into sub-solutions → components	Lowest-level components designed first → combined into next abstraction level
Used when	Design is to be formed from scratch / ground level	An existing software system exists and new system formed based on it
Example	Online Shopping → User module → Product → Cart → Payment → Delivery	Database Access + Utility → Reports → User Interface

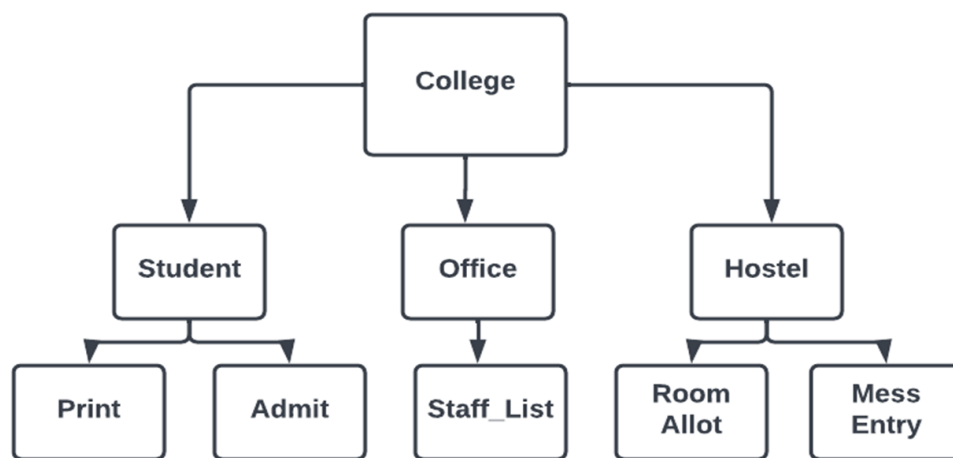


Figure: Top-Down Design — College system decomposed into Student, Office, Hostel modules

Functional vs Object-Oriented Design

Aspect	Functional Design	OOD
Focus	Functions with dedicated tasks	Objects and classes
Primary elements	Variables and functions (cannot be altered)	Objects and methods (can change data)
Priority	Algorithm/method more important than data	Data more important; method can change based on data
Data hiding	Not possible	Possible via private/protected access

Data Flow Diagrams (DFDs)

DFD DEFINITION

A graphical representation showing how data flows through a system, where it is stored, and how it is processed. Uses four symbols: External Entities, Processes, Data Flows (arrows), and Data Stores.

DFD Level	Description
Level 0 (Context Diagram)	Presents overview of system as single process with external entities; bird's-eye view
Level 1	Breaks main process into sub-processes; shows internal data flow in more detail
Level 2+	Further breakdown of Level 1 processes; used for complex systems only

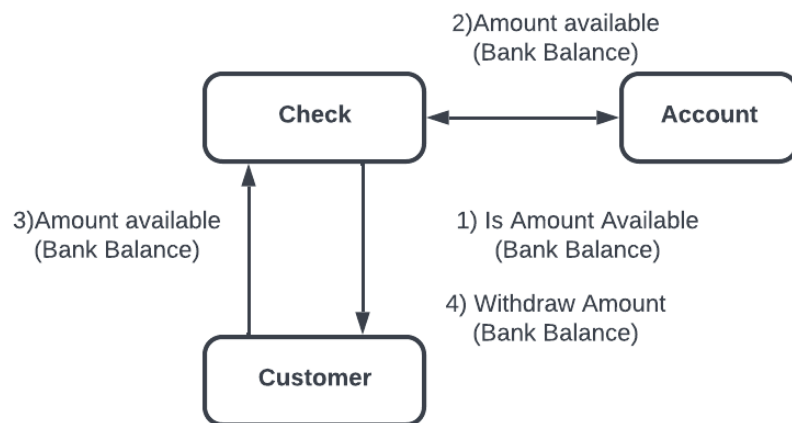


Figure: DFD Example — data flow between processes and data stores

2.7 Cohesion

COHESION

A measure of the functional strength of a module. Higher cohesion = better design. It groups things that make sense together.

Type	Description	Example
Logical	All elements perform similar operations	print() printing student attendance AND teacher attendance AND marks
Temporal	All functions execute at the same time	Start, shutdown, sleep, initialize of a process
Procedural	All functions are part of the same algorithm	Enter user ID, password, captcha — parts of authorization goal
Communicational	All functions refer to or update the same data structure	add(), delete(), print(), modify() working on STUDENT database
Sequential	Output of one element is input to the next	Insert card → Enter PIN → Authorize — ATM functions
Functional ■ (Best)	All elements work together to achieve a single task	seat_allotment module — select class, choose window seat preferences

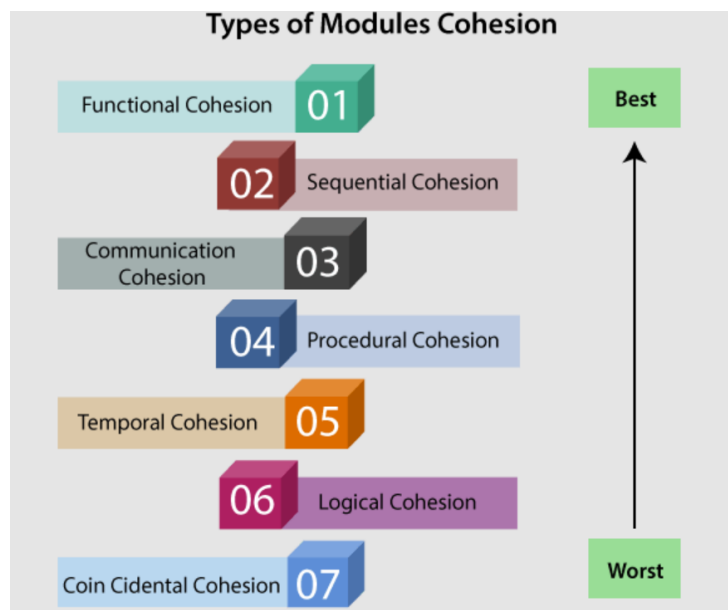


Figure: Types of Cohesion — ranked from Functional (Best) to Coincidental (Worst)

★ EXAM RULE

Functional Cohesion is the MOST DESIRABLE. High Cohesion = Good Design.

2.8 Coupling

COUPLING

A measure of the interaction between two modules. Lower coupling = better design. Good software should have as independent modules as possible.

Type	Description	Example
Data ■ (Best)	Two modules communicate using a common parameter (simple data)	Bank application passing account number between modules
Stamp	Two or more modules use composite data (entire data structure)	Struct in C or classes/objects in C++ passed between modules
Control	Data from one module used to direct instruction order in another	draw() passes "circle"/"square" command to control which function runs
Common	Two modules share information through global data items	Student management modules sharing a global STUDENT structure
Content ■ (Worst)	More than one module share code; branch from one to another	menu() directly calling add(), subtract(), mult() — direct code sharing

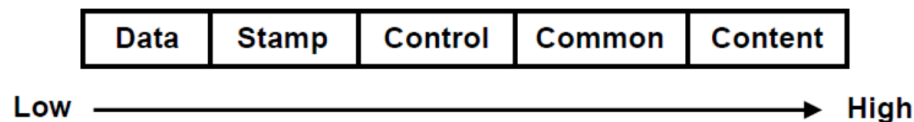


Figure: Coupling Scale — Data (lowest/best) to Content (highest/worst)

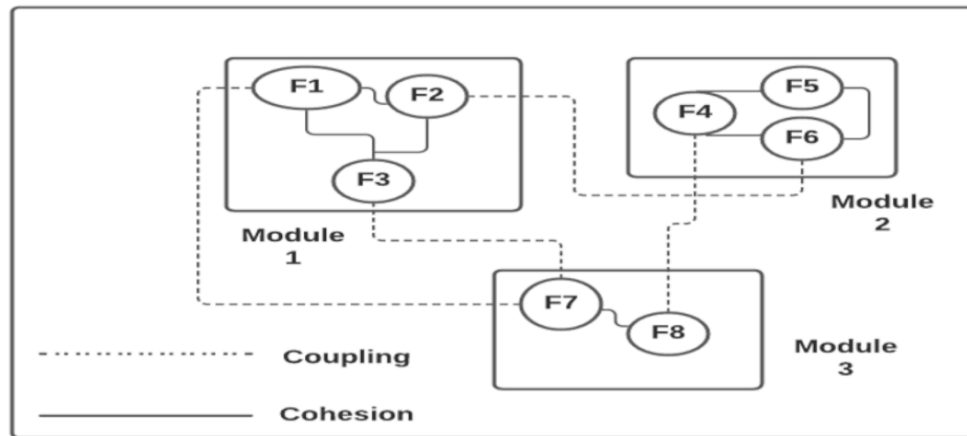


Figure: Cohesion vs Coupling — modules with high internal cohesion and low inter-module coupling

★ GOLDEN RULE

HIGH COHESION + LOW COUPLING = GOOD DESIGN. This leads to better error isolation, more reuse, and improved understandability.

2.9 UML Diagrams

Unified Modeling Language (UML) is a general-purpose visual modeling language. UML is NOT a programming language — it is a visual language for designing systems.

Diagram	Category	Purpose
Class Diagram	Structural	Shows classes, attributes, methods, and relationships between classes
Object Diagram	Structural	Shows real object instances with actual values at a specific point in time
Component Diagram	Structural	Shows how physical components in a system are organized
Deployment Diagram	Structural	Shows system hardware and its software; represents system architecture
Use Case Diagram	Behavioral	Shows user interaction with system (actors and use cases); high-level view
Activity Diagram	Behavioral	Shows step-by-step flow of activities; like a flowchart
Sequence Diagram	Behavioral	Shows message flow between objects over time; order of interactions

Diagram	Category	Purpose
State Diagram	Behavioral	Shows states of an object and transitions between states

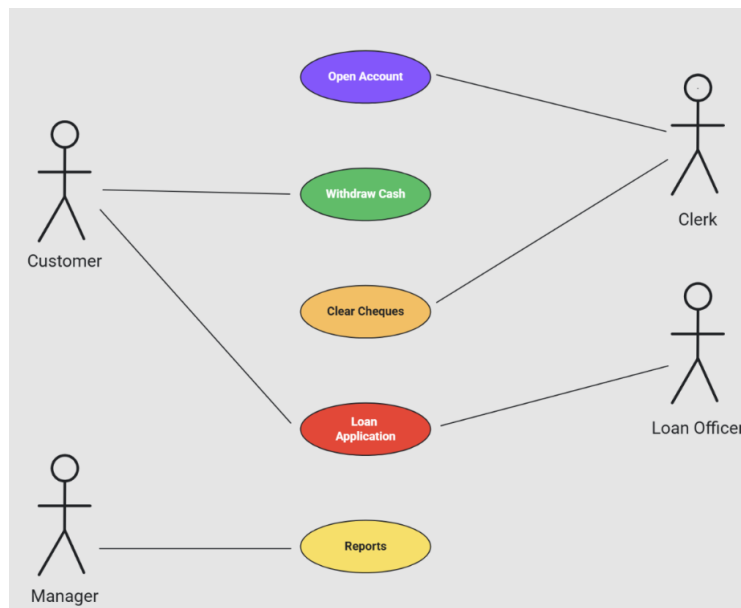


Figure: UML Use Case Diagram — Banking system with Customer, Clerk, and Loan Officer actors

Class Relationships

Relationship	Symbol	Description	Example
Association	Solid line	Simple connection between two classes	Student ↔ Teacher
Aggregation	Solid line + hollow diamond	"Has-a"; contained objects CAN exist independently	Person ↔ Pet
Composition	Solid line + filled diamond	"Has-a" (stronger); contained object CANNOT exist without container	Person ↔ Address
Inheritance	Solid line + hollow arrowhead to parent	Subclass inherits from superclass	Dog, Cat inherit from Animal



Software Coding

8 Contact Hours · Guidelines · Documentation · Code Review



3.1 Introduction to Coding

DEFINITION

Software Implementation/Development/Coding is the process of converting the design of a system into a high-level language program. It results in an executable version of the software. Coding guidelines are defined by the organization; different tools (compilers, debuggers, interpreters) are used.

Important Aspects to Consider

- For what application is the software being used?
- Use of new technology; time necessary for software launch
- Scalability, Security features, Tool availability, Efficiency
- Reuse of the software (COTS — Commercial Off-The-Shelf)
- Managing configuration of the software

Software Reuse — Levels

COTS (Commercial Off-The-Shelf) components are readily available, can be bought and modified. Reuse reduces time and cost.

Level	What is Reused	Example
Abstraction Level	Architecture and design patterns reused	MVC pattern reused across projects
Component Level	Front ends, GUI, tables, database structures	Reusing login component
Object Level	Existing library files, objects, and files	Java Collections library
System Level	Entire existing system modified and refined	Student record software modified for a new college

Configuration Management

A process of managing changes during software development when a team of people is working. Three main activities:

- **Version Management** — keeping track of different versions of software
- **System Integration** — support for combining different version components into a single system

- **Problem Tracking** — tools for error detection and correction

3.2 Coding Guidelines

Coding guidelines provide common knowledge for using coding style. The actual implementation is up to the individual, but guidelines ensure uniformity and quality. If coding standard is not maintained, code might get rejected in code review.

- **Size:** Code should be small. Functions > 10–15 lines defeat the purpose of modularity and make error detection very challenging.
- **Variable Scope:** Clarify global vs. local variables (block/file scope) to limit access to different users.
- **Module Documentation:** Include module name, author, date, brief synopsis, modification details.
- **Naming Conventions:** Variables: alphabets, digits, underscores (e.g., `Student_Name`). Constants: all capitals (e.g., `MAX`).
- **Error Handling:** Return 1 or 0 based on conditions — uniform, reusable style.
- **Avoid go-to statements:** Unconditional jumps make programs difficult to understand and debug.
- **Meaningful variable names:** Use `temp_loan_value` over just `temp`.
- **Side effects:** A side effect is modification of parameters outside a function's definition. I/O operations or pass-by-reference may create ambiguous alterations.

3.3 Code Documentation

PURPOSE

Documentation connects humans and machines. Helps developers working on long-term projects, allows others to continue work, shows contributions, and promotes open-source transparency.

Major Steps in Software Documentation:

1. README file — installation procedure, OS, hardware requirements, sample usage
2. Module functionality — parameters, their types, and return values for each module
3. Naming conventions — files, variables, constants, programming practices, comments
4. Contributors — name contributors in a team with proper citations
5. License and version — license details and version of the software
6. Developer roles — names and email addresses for future contact
7. Version history — current version and editing performed

3.4 Code Review

CODE REVIEW

Done after compilation of a coding module. Detects algorithmic, logical, and poor programming issues. Does NOT check syntax errors. Different from testing — review is on non-executable code.

Walkthrough vs Inspection

Aspect	Code Walkthrough	Code Inspection
Purpose	Find algorithmic and logical errors	Find common programming errors (does not execute code)
Process	Each team member traces code manually using sample test cases	Reviews programming practice, algorithm choice, library usage, coding standard
Presence	Done in the presence of the coder who developed the code	Independent review; coder may not be present
Formality	Informal — relies on experience, knowledge, common sense	More formal — checks specific error categories
What is found	Whether code is working and producing desired results	Infinite loops, memory allocation, type mismatch, wrong operator precedence, dangling pointers
Output	Minutes of walkthrough recorded and used as coding guidelines	List of specific errors and coding standard violations

★ KEY DIFFERENCE

Walkthrough: Code execution performed (with test data) to check results. Inspection: No execution — reviews the way of programming and looks for known error patterns.

UNIT
4

Testing & Software Project Management

9 Contact Hours · White/Black Box · COCOMO · Risk Management



4.1 Testing Fundamentals

TWO GOALS OF TESTING

1. Illustrate that the developed software fulfills client requirements. 2. Search for scenarios and data where software may not work as per specification or may result in an error or corrupt data.

Key Terminology

Term	Definition	Example
Error	A human mistake made during software development (wrong design decision, wrong logic)	Developer writes $\text{area} = \text{length} + \text{breadth}$ instead of $\text{length} * \text{breadth}$
Fault / Bug / Defect	Manifestation of an error in the software artifact	Missing validation for division by zero; incorrect database schema
Failure	System doesn't perform as expected; produces incorrect/unexpected results	Banking app calculates interest incorrectly; game freezes at level 50

Test Case: A triplet [I, S, O] — I = input data, S = system state at input time, O = expected output. A **Test Suite** = set of all test cases. **SUT** = Software Under Test.

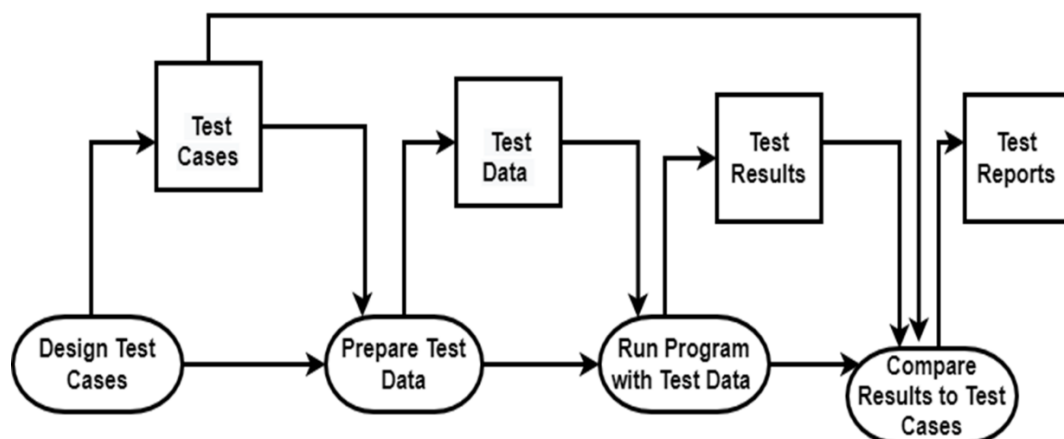


Figure: Testing Process Flow — from Design Test Cases to comparing results

4.2 Verification vs Validation

Aspect	Verification	Validation
Question	"Are we building the product right?"	"Are we building the right product?"
Code execution	May NOT include execution — static tests	Always involves execution of code
Performed by	System analysts and designers	Software development team
Techniques	Code inspections, reviews, walkthroughs	Unit testing, integration testing, system testing

4.3 Levels of Testing

Unit Testing

Done in the coding phase. Tests the smallest element of software (module) in isolation. Each module tested separately before combining. Helps correct errors at early stage.

- Checking loops, functions working in a program
- Checking initialization values, operator precedence

Integration Testing

Different modules integrated using a software plan; checks the steps and sequence by which different modules form a single system.

Approach	Description	Notes
Top-Down	Higher/main modules tested first; lower modules use stubs (dummy programs substituting for lower modules)	Main control tested early; stubs for incomplete modules
Bottom-Up	Lower modules tested first; drivers used as dummy main modules if higher modules not ready	Detailed functionality tested first
Big-Bang	All modules simply put together and tested at once	Useful for small systems; error handling very difficult; high-risk modules not prioritized
Sandwich/Mixed	Combination of top-down and bottom-up; tests as modules become available	Used in large systems; expensive; not suitable for high coupling

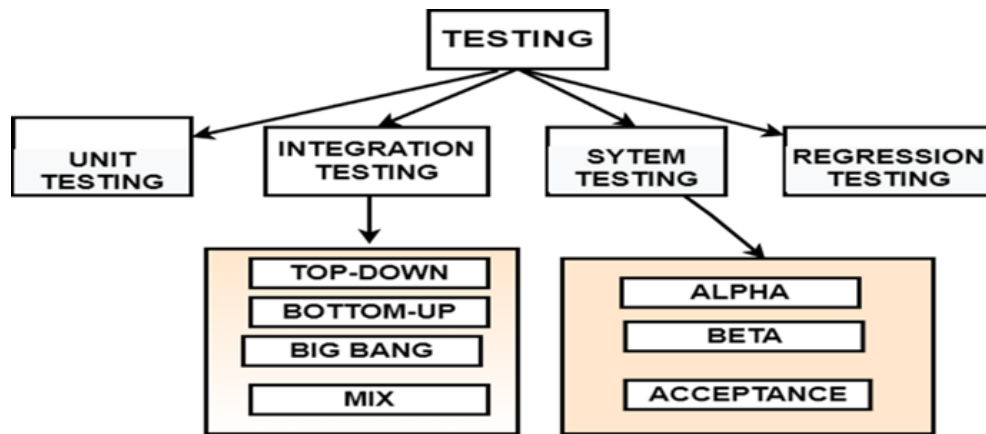


Figure: Testing Levels — from Unit Testing through Integration to System and Acceptance Testing

System Testing & Acceptance Testing

Type	Description	Example
Alpha Testing	Done by internal team/high-level developers in early stage before release	Google releasing new feature to high-level developers first
Beta Testing	Beta version given to limited users to test and improve quality; users raise problems for developers	Software company releases to few small colleges for feedback
Acceptance Testing	End-users test the system and decide if it's ready for deployment; last phase; payment done after this	User Acceptance Testing (UAT)
Regression Testing	Testing when an existing system is changed or upgraded; checks for new bugs after new features	Performed in the maintenance stage

4.4 White Box Testing

WHITE BOX TESTING

Also known as clear box / glass box / structural testing. Tests the internal logic, structure, and flow of the code. Tester understands the code and inner workings. Performed by the software developer.

Method	Description	Note
Statement Coverage	Every statement executed at least once; checks for unused statements, missing branches	May work properly for one set but not for others
Branch Coverage	Each conditional branch tested for true and false values; stronger than statement coverage	Guarantees all test cases fulfilled
Condition Testing	Used when there are composite conditions; for n expressions, 2^n test cases required	Exponential growth in test cases
Path Testing	Tests control paths depending on if, if-else, loops; requires complete knowledge of program	Values of different variables can be traced

Method	Description	Note
Data-flow Testing	Uses definitions and uses of variables as test paths; points out undeclared variables	Good for detecting variable-related bugs
Mutation Testing	Statements of source program changed (mutated); if output differs from original, mutant is killed	Changes: data type, constant value, operator order

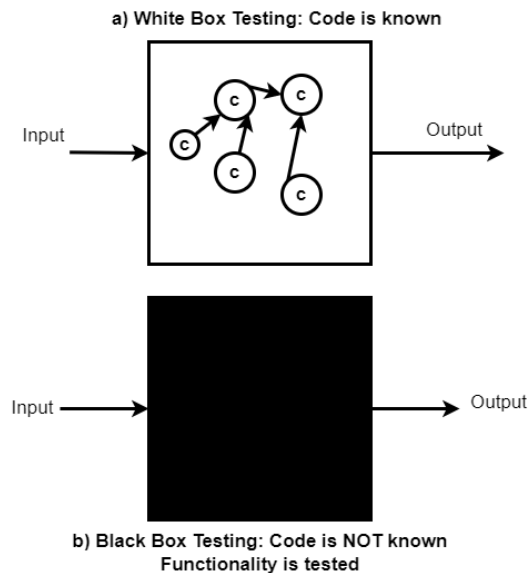


Figure: White Box vs Black Box Testing — glass box with visible paths vs opaque black box

4.5 Black Box Testing

BLACK BOX TESTING

Also known as functional / behavioral testing. Tests functionalities using requirement specifications without knowledge of design or code. Tests behavior visible from outside. E.g., testing websites like gmail.com, irctc.in.

Techniques

Equivalence Partitioning:

Input values partitioned into equivalence classes where the program behaves similarly for every input in the class. Test with one representative from each class.

EXAMPLE — SQRT (1 TO 5000)

Classes: {negative integers} [invalid], {1–5000} [valid], {integers > 5000} [invalid]. Test suite: {-5, 500, 6000}

Boundary Value Analysis (BVA):

Typical programming errors occur at **boundaries** of equivalence classes (programmers use $<$ instead of $\leq$). Select test cases AT the boundaries.

For SQRT (1–5000): test values {0, 1, 5000, 5001}. For age eligibility (18–60): test values {17, 18, 60, 61}.

Other Techniques:

- **Decision Table Testing** — conditions mentioned in decision tables; good for complex business rules
- **State Transition Testing** — tests transitions of states from one form to another

Aspect	White Box	Black Box
Knowledge needed	Internal code and logic	Only functional specifications
Performed by	Software developer	Test engineer or user
What is tested	Code structure, paths, coverage	Functionality and behavior
Also called	Clear/Glass box, Structural testing	Functional/Behavioral testing

4.6 Software Project Management (SPM)

DEFINITION

SPM is managed by the project manager within the allocated budget and time constraints. The project manager takes responsibility for delivering a project that meets client requirements and is of high quality.

SPM Goals

- Maintain the agreed timing of software delivery
- Keep project activities under the allotted budget
- Fulfill the client's specifications
- Make sure the development team is functioning well in a positive atmosphere

Project Planning Estimates

Parameter	Description
Size	Complexity in terms of efforts and time required (LOC, Function Points)
Cost	Finance necessary to complete the project
Duration	Time required for completion
Efforts	Manpower and other resources required for project completion

4.7 Size Estimation — LOC & Function Points

Lines of Code (LOC)

Simplest metric — counts number of source instructions. Comments and header file statements are excluded. Experience and skill level of the developer is considered.

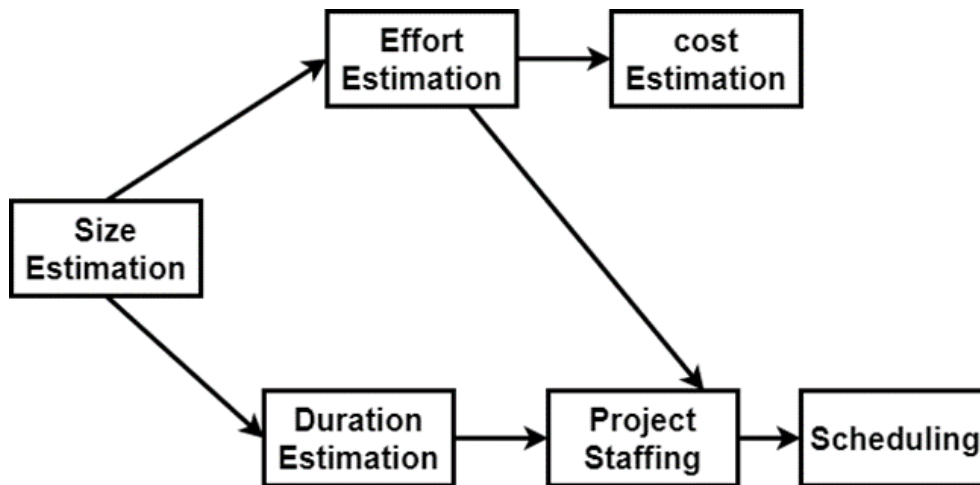


Figure: LOC Estimation — project size estimation approach with skill levels

Expected LOC: $S = (S_{opt} + 4 * S_m + S_{pess}) / 6$

Where: S_{opt} = Optimistic (efficient developer)

S_m = Most Likely (average developer)

S_{pess} = Pessimistic (poor developer)

Productivity = $LOC / \text{Effort (person-months)}$

Staffing = $\text{Effort} / \text{Duration}$

Cost = $\text{Effort} * \text{Cost_per_person_month}$

WORKED EXAMPLE

$S_{opt}=5200$, $S_m=7000$, $S_{pess}=9000$. $S = (5200 + 4 \times 7000 + 9000)/6 = 7033$ LOC. With 2-person team: 24 person-months. Productivity = $7033/24 = 293$ LOC/person/month. Duration=6 months: Manpower = $24/6 = 4$ persons. Cost at Rs.2500/PM: $24 \times 2500 = \text{Rs.}60,000$.

Function Point (FP)

Proposed in 1983. Measures size based on **functional view** — considers: number of inputs, outputs, queries, internal files, and external files. Overcomes many pitfalls of LOC. Can be used from problem specification stage.

- Functionality measured based on user's request and response
- Cost estimation independent of technology
- Requirements can be verified using function count
- Drawback: uses much judgment which may not be accurate; time-consuming

4.8 COCOMO Model**COCOMO — CONSTRUCTIVE COST MODEL**

Proposed by Barry Boehm in 1981 after studying 63 projects. Based on LOC. Estimates size, effort, cost, time, and quality.

Three Project Categories

Category	Size	Description	Example
Organic	2–50 KLOC	Well-understood; small team with good experience; simple problems	Batch data processing, payroll
Semidetached	50–300 KLOC	Mix of experienced and inexperienced team; utility programs	Transaction systems, compilers
Embedded	> 300 KLOC	Strongly connected to complex hardware; complex projects with restricted regulations	Real-time control systems, avionics

Basic COCOMO Model

```

Effort:    PM = a1 * (KLOC)^b1    [Person-Months]
Dev Time:  T  = a2 * (PM)^b2      [Months]
Staffing:  N  = PM / T            [Persons]
Productivity = KLOC / PM          [KLOC/Person-Month]

```

Mode	a1	b1	a2	b2
Organic	2.4	1.05	2.5	0.38
Semidetached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

Project	a1	a2	b1	b2
Organic	2.4	1.05	2.5	0.38
Semi Detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

Figure: COCOMO Constants — a1, b1, a2, b2 for Organic, Semidetached, and Embedded modes

Intermediate COCOMO Model

The Basic model only uses LOC. Intermediate model introduces **cost drivers (multipliers)** — Experience & capability of team, Reliability requirements, Hardware performance constraints, Use of software tools.

Effort = Basic Effort × EAF (EAF = Effort Adjustment Factor = product of all cost driver multipliers)

Complete (Detailed) COCOMO Model

Most large systems have many smaller sub-systems with different types (organic, semidetached, embedded). Complete COCOMO considers each subsystem separately and estimates effort for each.

- Total effort = sum of estimates for all individual subsystems
- Applied in each phase of software engineering
- Six phases: Planning & requirements, System design, Detailed design, Module code & test, Integration & test, Cost Constructive model

COMPLETE COCOMO EXAMPLE

Distributed MIS: GUI (organic) + Database (semidetached) + Communication (embedded). Total cost = sum of individual sub-system estimates using their respective COCOMO constants.



Quality, Maintenance & CASE Tools

9 Contact Hours · CMM · ISO 9000 · SCM · CASE Tools



5.1 Software Quality Concepts

QUALITY SOFTWARE

Defect-free software, delivered within the allocated budget and time, meets the client's requirements, and can be easily maintained. Just meeting specifications is not enough — if it works but is slow and uses excessive memory, it is NOT quality software.

Three Levels of Quality Concerns

- **Organizational Level:** Process and standards defined by the organization for quality (collecting requirements, design standards, code standards)
- **Project Level:** Quality processes applied to meet standards; quality plan developed with project goals
- **Individual Level:** Adherence to coding and documentation standards

Quality Plan Structure

- **Product Introduction:** Features, benefits, specifications, target audience
- **Product Plan:** Strategy and tactics for launching and supporting the product
- **Process Description:** Development and service processes used
- **Quality Goals:** SMART goals aligned with project objectives
- **Risk Management:** Key risks (technical, schedule, cost, quality) and mitigation

5.2 Software Quality Attributes

Attribute	Description
Safety and Security	No harm caused; preventing unauthorized access/illegal activity
Reliability	Performs intended functions without errors or failures consistently
Portability	Works on different OS environments, systems, platforms
Usability	Both expert seniors and new users can use without difficulty
Reusability	Different components can be reused to develop new software
Modularity	Divided into smaller modules; developers can work in parallel

Attribute	Description
Maintainability	Easily corrected; new functions added without much overhead

Software Standards

Type	Description	Example
Process Standards	Define good software development processes — SDLC stages, tools, documentation	Design review conduct, version release process
Product Standards	Applied during development of software product — coding standards, documentation standards	Design review form, Java Programming Style, Project Plan Format

5.3 ISO 9000 Certification

ISO 9000

International Standards Organization (ISO) published its 9000 series in 1987. Provides guidelines for maintaining quality and serves as a reference for contracts. ISO 9000 gives guidelines for PRODUCING the product — NOT concerned with the product itself.

Types of ISO 9000

Standard	Applies to
ISO 9001	Design, development, production, and servicing — applies to MOST software organizations
ISO 9002	Organizations NOT involved in design but only production (car/steel); does NOT apply to software
ISO 9003	Organizations involved ONLY in installation and testing of products

Why ISO Certification?

- Customer gets confidence in the product
- Well-structured and documented process assures high quality
- Development process is more efficient and cost-effective
- Weak points identified and remedies given
- Sets basic framework for efficient process → Total Quality Management (TQM)

Limitations:

- Sets steps for production process but does NOT guarantee high quality
- Not a foolproof agency; may downplay domain experts
- Heavy emphasis on documentation — takes time and effort

5.4 SEI Capability Maturity Model (CMM)

SEI CMM

Used by the Software Engineering Institute to improve software quality. Two uses: (1) Capability Evaluation — assess software process capability of an organization; (2) Software Process Assessment — improve process capability.

Five Maturity Levels

Level	Focus	Key Process Areas (KPA)
1 — Initial	Competent people; ad-hoc processes; unpredictable results; chaotic	None
2 — Repeatable	Project Management; basic processes established; can repeat successes	Software project planning, configuration management
3 — Defined	Defining of processes; organization-wide standards; documented	Process definition, training programs, peer reviews
4 — Managed	Product and process quality; quantitative control; measured	Software metrics and quality management
5 — Optimizing	Continuous process improvement; defect prevention; innovation	Defect prevention, process change management, technology change management

★ CMM VS ISO

ISO 9000 = Certification-based (Yes/No compliance); focused on documentation; international standard. CMM = Maturity-level based (1–5); focused on CONTINUOUS process improvement; specifically for software; developed by SEI.

5.5 Software Maintenance

DEFINITION

Software maintenance is a general process of modifying a software product after it is delivered to the client. Modifications are necessary to correct coding bugs, accommodate major design changes, and manage new requirements. Development does NOT end after delivery.

Types of Software Maintenance

Type	Description	Trigger
Corrective	Corrects faults and errors reported by customers after system is put in use. Errors may be in requirements, design, or code.	Customer bug reports; system failures
Adaptive	Used when software environment changes — new OS, new web/mobile platform, new hardware added.	Environment changes (new OS, new hardware)
Perfective	Advancing requirements; adding new features user realizes after delivery; removing unwanted features; enhancing UX.	User requests new features; system improvement
Preventive	Future problems prevented using modifications and updates; helps increase software lifetime; code optimized for future.	Proactive improvement; stability and maintainability

5.6 Forward, Reverse Engineering & Re-engineering

Forward Engineering

Standard software development approach — follows steps in order: Requirement Gathering → Preparing Software Design → Coding → Testing → Maintenance

Reverse Engineering

DEFINITION

The process of recovering the design, requirement specifications, and functions of a product from an analysis of its code. Used for legacy systems that are not well-structured or lack documentation.

Steps in Reverse Engineering:

1. Code Improvement — improve readability; give meaningful names; simplify complex statements; replace nested loops. ("Cosmetic changes" — functionality NOT changed.)
2. Code Analysis — complete analysis of modules and their interfaces
3. Design Extraction — extract design using tools (structure charts, DFDs, control flow diagrams)
4. SRS Writing — write the SRS document based on the extracted design

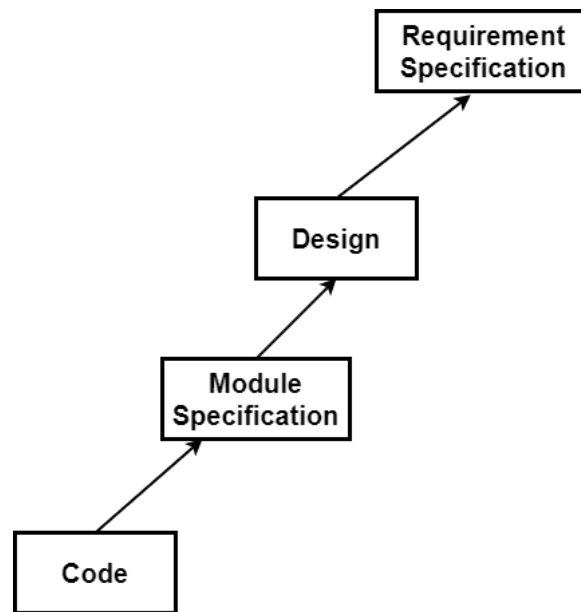


Figure: Reverse Engineering — steps from code analysis back to requirement specification

Software Re-engineering

DEFINITION

A combination of reverse engineering followed by forward engineering. Legacy products are reverse-engineered to get module specifications, which are used to generate design, then new requirements are added and built with forward engineering.

Legacy Code → Reverse Engineering → Module Specifications

↓

New Requirements Added → Forward Engineering → Improved Product

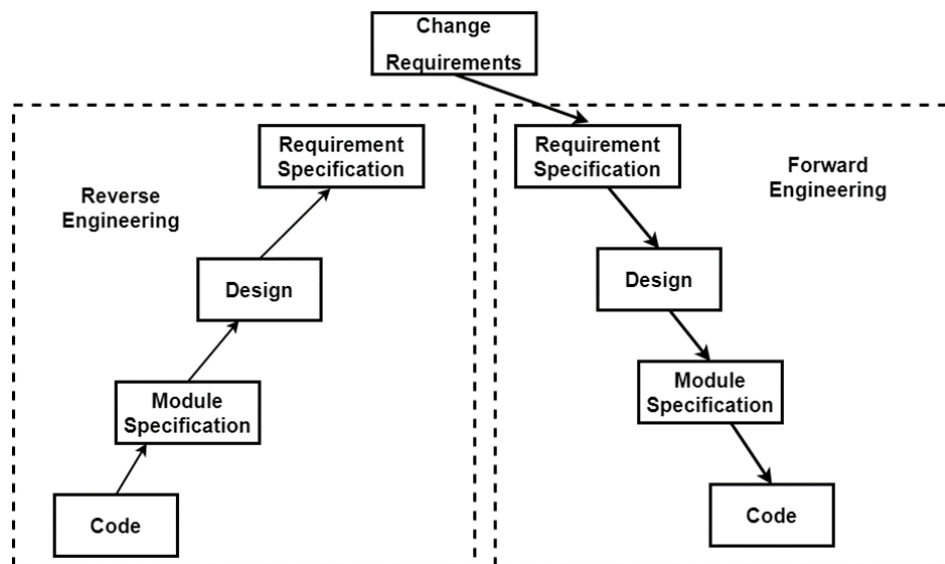


Figure: Re-engineering — Reverse Engineering combined with Forward Engineering via Change Requirements

Aspect	Reverse Engineering	Re-engineering
Direction	Code → Design → Requirements	Code → Design → Requirements → Code (combined)
Purpose	Understand/document an existing system	Transform an existing system into a new improved one
Output	Documentation, design	Improved software product with new requirements

5.7 Software Configuration Management (SCM)

SCM DEFINITION

SCM maintains the status of the software product and keeps account of changes. Configuration refers to the state of source code, design, SRS, test reports, user manuals used by software engineers during SDLC.

Types of Changes

- **Release:** New bug fix, functionality, or small technology changes. Example: Windows patches.
- **Version:** Major change in hardware/technology or drastically changed functionality. Example: Android updates, Windows 95 → XP → 10.
- **Revision:** Minor bugs or errors. Example: MS Office minor typesetting revisions.

SCM Process Activities

1. Configuration Identification — objects classified into controlled, pre-controlled, and uncontrolled
2. Version Control — tools and methods to create different versions of objects
3. Change Control — requests for variation received; impact analyzed; Change Control Authority (CCA) authorizes or rejects changes; access control and synchronization control policies

4. Configuration Audit — checks whether original requirements are fulfilled in new deliverable; generates audit reports
5. Status Reporting — generates reports on current configuration status for all stakeholders

5.8 CASE Tools

CASE TOOLS DEFINITION

CASE tools provide automated support in software engineering for ALL activities: gathering requirements, preparing software design, developing code, testing, configuration, and project management. Different from a programming environment — covers ALL phases of SDLC.

Types of CASE Tools

Type	SDLC Phase Covered	Examples
Upper CASE	Requirement collection, Planning, Analysis, Design — Early stages	Accept 360, CASE Complete
Lower CASE	Coding, Testing, Maintenance — Later stages	Eclipse, Cscope, SoapTest
Integrated CASE	All stages including documentation	Rational Rose, Visual Paradigm

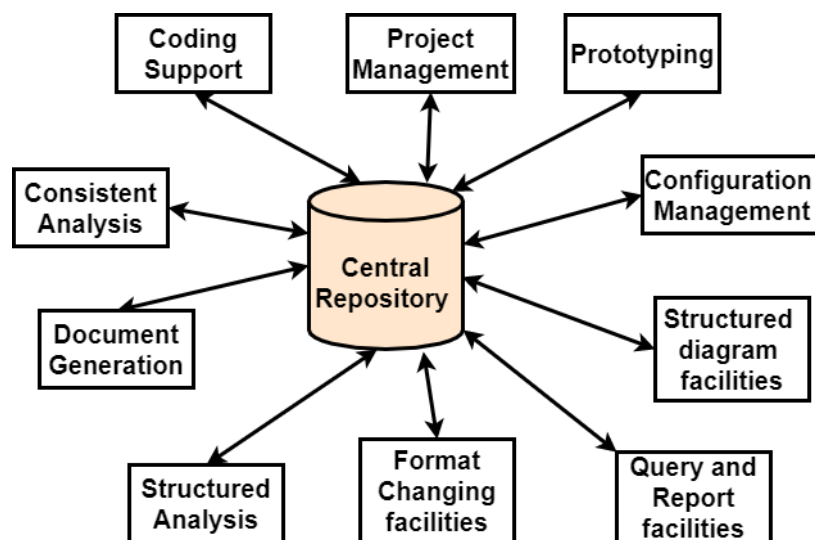


Figure: CASE Tools Classification — Upper, Lower, and Integrated CASE tools across SDLC phases

Advantages and Limitations:

- Cost savings of 30–40% at different phases of SDLC
- Better quality — already tested components
- Consistent documentation generated; fewer chances of human error
- Systematic and consistent SDLC phases
- Staff must be trained — resistance to change is common
- Project size may not suit certain CASE tools

- ■ Quality level may not be desirable for certain applications



Practice Problems & Solved Assignments

COCOMO · CPM · Test Cases · Design Questions



COCOMO Solved Problems

Q1 — 32 KLOC, Organic Mode ($a=2.4$, $b=1.05$, $c=2.5$, $d=0.38$)

$$\begin{aligned}\text{Effort (PM)} &= 2.4 \times (32)^{1.05} \\ &= 2.4 \times 37.88 \\ &= 90.9 \text{ Person-Months}\end{aligned}$$

$$\begin{aligned}\text{Dev Time} &= 2.5 \times (90.9)^{0.38} \\ &= 2.5 \times 9.22 \\ &= 23.05 \text{ Months}\end{aligned}$$

$$\text{Avg Staff} = \text{PM} / \text{T} = 90.9 / 23.05 = 3.94 = 4 \text{ persons}$$

Q2 — 50 KLOC, Semidetached ($a=3.0$, $b=1.12$, $c=2.5$, $d=0.35$)

```
Effort (PM) = 3.0 x (50)^1.12
            = 3.0 x 76.99
            = 231 Person-Months
```

```
Dev Time   = 2.5 x (231)^0.35
            = 2.5 x 9.36
            = 23.4 Months
```

```
Productivity = KLOC / PM = 50 / 231 = 0.216 KLOC/PM
```

Q3 — *Project A (20 KLOC) vs Project B (60 KLOC), Organic*

```
Project A: PM_A = 2.4 x (20)^1.05 = 2.4 x 22.97 = 55.1 PM
            Productivity_A = 20 / 55.1 = 0.363 KLOC/PM
```

```
Project B: PM_B = 2.4 x (60)^1.05 = 2.4 x 71.46 = 171.5 PM
            Productivity_B = 60 / 171.5 = 0.350 KLOC/PM
```

Conclusion: Project A has BETTER productivity (smaller = more productive per person)

Critical Path Method (CPM)

Given Activities

Activity	Duration	Predecessor
A	3	—
B	2	A
C	4	A
D	6	B, C

FORWARD PASS (ES to EF):

A: ES=0, EF=0+3=3
B: ES=3, EF=3+2=5 [predecessor A, EF=3]
C: ES=3, EF=3+4=7 [predecessor A, EF=3]
D: ES=7, EF=7+6=13 [predecessors B[EF=5], C[EF=7] -> max=7]

BACKWARD PASS (LF to LS):

D: LF=13, LS=13-6=7
C: LF=7, LS=7-4=3
B: LF=7, LS=7-2=5
A: LF=min(3,3)=3, LS=3-3=0

SLACK = LS - ES:

A: Slack=0 <- CRITICAL B: Slack=2
C: Slack=0 <- CRITICAL D: Slack=0 <- CRITICAL

Critical Path: A -> C -> D Project Duration: 13 units

Equivalence Partitioning & BVA — Test Case Tables

Case Study 1: Age Eligibility (18–60)

Equivalence Classes: Class 1 (Invalid — too young: age < 18) · Class 2 (Valid: $18 \leq \text{age} \leq 60$) · Class 3 (Invalid — too old: age > 60) · Class 4 (Invalid — non-integer)

Test Value	Class	Valid/Invalid	Expected Output
17	Class 1 (Invalid)	Invalid	Reject — Too Young
18	Class 2 (BVA)	Valid	Accept
30	Class 2	Valid	Accept
60	Class 2 (BVA)	Valid	Accept
61	Class 3 (Invalid)	Invalid	Reject — Too Old
-5	Class 1 (Invalid)	Invalid	Reject — Invalid Input
25.5	Class 4 (Invalid)	Invalid	Reject — Non-integer

Case Study 2: Online Payment Gateway

Amount: Rs.10 – Rs.50,000. If Amount > Rs.25,000, OTP mandatory. Payment Method: {Card, UPI, NetBanking}.

Test Case	Amount	Method	OTP	Expected
TC1 — Valid, no OTP	Rs.500	Card	N/A	Success
TC2 — Valid, OTP required	Rs.30,000	Card	Provided	Success
TC3 — OTP missing	Rs.30,000	NetBanking	Not provided	OTP Required
TC4 — Invalid method	Rs.1,000	Cash	N/A	Invalid Payment Method
TC5 — BVA: Rs.10	Rs.10	Card	N/A	Success
TC6 — BVA: Rs.50,000	Rs.50,000	Card	Provided	Success
TC7 — Below min	Rs.5	UPI	N/A	Invalid Input
TC8 — Above max	Rs.60,000	Card	N/A	Invalid Input

Feasibility Study & Design Principles (Assignment 1)

Q. Importance of Feasibility Study

- Determine if the project is technically achievable within existing technology
- Assess whether the project can be completed within the allocated budget
- Verify if the project aligns with organizational objectives
- Identify risks early — saving time, money, and effort
- Provide a binary Yes/No decision to prevent investment in infeasible projects

- Ensure legal compliance and schedule adherence

Q. Key Design Principles with Application Examples

Principle	Definition	Application Example
Modularity	Divide system into smaller, manageable modules	Online Shopping: Login, Cart, Payment, Order — each separate module
Abstraction	Focus on what a component does, not how it does it	User calls process_payment() without knowing internal bank API calls
Information Hiding	Internal details of a module should not be exposed	Payment module exposes only "pay(amount, method)" — hides encryption
Low Coupling	Modules should be as independent as possible	Cart module uses an API layer instead of directly accessing database
High Cohesion	Module performs a single, well-defined task	seat_allotment module handles only seat selection and preferences

End of Software Engineering Examination Preparation Guide · All 5 Units · TMC 202

Graphic Era Hill University · Department of Computer Applications